

มหาวิทยาลัยมหิดล
Mahidol University

ห้ามมิให้นักศึกษาทำซ้ำ ดัดแปลง หรือใช้ประโยชน์จากงานอันมีลิขสิทธิ์ปรากฏอยู่ในระบบการศึกษาอิเล็กทรอนิกส์ (E-Learning) ของมหาวิทยาลัย ไม่ว่าจะทั้งหมดหรือบางส่วน โดยไม่ได้รับอนุญาตจากมหาวิทยาลัย นอกเหนือจากการศึกษาส่วนบุคคล ทั้งนี้ การทำซ้ำ ดัดแปลง หรือเผยแพร่ต่อสาธารณชนซึ่งงานอันมีลิขสิทธิ์ **จะมีโทษปรับตั้งแต่ 20,000 บาท ถึง 200,000 บาท และหากเป็นการกระทำเพื่อการค้า จะมีโทษจำคุกตั้งแต่ 6 เดือน ถึง 4 ปี หรือปรับตั้งแต่ 100,000 บาท ถึง 800,000 บาท หรือทั้งจำทั้งปรับ**

Unless allowed by Mahidol University, the User shall not copy, modify, or exploit in part or in whole of the copyrighted materials on the Platform, other than for your own individual study. The copy, modify or communication to public of the copyrighted materials shall be inflicted with a fine from 20,000 Baht up to 200,000 Baht. If the offence is committed with the commercial purpose, the offender shall be inflicted with imprisonment for a term from 6 months up to 4 years or a fine from 100,000 Baht up to 800,000 Baht or both imprisonment and fine.

Mahidol University
Faculty of Information
and Communication Technology



ITDS261 Introduction to Software Engineering

Lecture 10 – Implementation & Code Review

Note: Some slides are adapted from COMPGS04 Tools and Environment, UCL (Dr. Jens Krinke)

Topics

- Programming Paradigms
- Implementation
- Code comments
- Code review

Programming Paradigms

- **Imperative programming**
  - Tell the computer *how* to do something step by step (e.g., C, Python, Java, JavaScript)
- **Declarative programming**
  - Specify *what* you want the program to accomplish, not *how*
  - *Example: SQL*
- **Object-oriented programming**
  - Model software as a collection of objects that have data (attributes) and behavior (methods) (e.g., Java, Python, C++, C#, Ruby)
- **Logic Programming**
  - Define *what* the problem is, not *how* to solve it
  - Example: Prolog

Some programming languages support multiple paradigms (i.e., *multi-paradigm* languages).

Emerging Development Approaches

- **No-code programming**
  - Allows users to create applications without writing any traditional programming code
  - Uses graphical interfaces, drag-and-drop components, and predefined logic blocks to build an app
  - Benefits:
    - Faster development
    - Less technical barrier
    - Easier experimentation and prototyping
  - Limitations
    - Limited flexibility for complex logic
    - Dependence on platform/tool
    - Harder to scale or customize deeply compared to traditional programming

Emerging Development Approaches (Cont.)

- **Low-code programming**
  - Allows users to build applications with minimal hand-coding
  - Uses visual tools (e.g., drag-and-drop components) with the option to write some code when needed.
  - Benefits:
    - Faster development than traditional coding
    - Good balance between ease of use and flexibility
  - Limitations
    - Requires some technical knowledge
    - Complex customizations can become hard to maintain
    - Dependence on platform/tool



https://www.mendix.com/

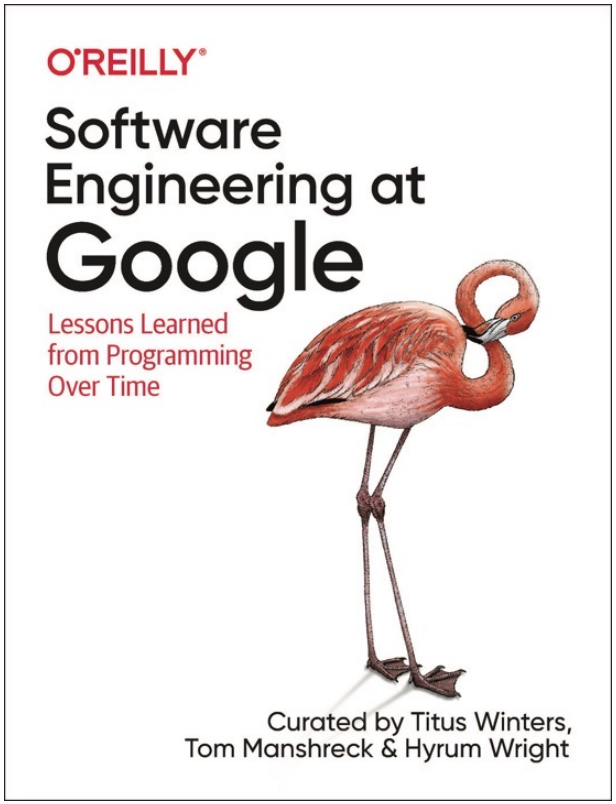
Emerging Development Approaches (Cont.)

- **Vibe coding**
  - Allows users to build programs by describing what they want using natural language
  - Users interact directly with an interface
  - Blends AI assistance, visual design, and live feedback
  - Benefits:
    - Less technical barrier
    - Interactive and fast development
  - Limitations
    - Limited flexibility for complex logic
    - Dependence on platform/tool
    - Overreliance on AI



Reference: Rapid Prototyping with AI-assisted Programming by Asst. Prof. Chaiyong Ragkhitwetsagul, Faculty of ICT, Mahidol University (21 July 2025)

References for Today (1)



- Content of coding styles, code review, and documentation in the lecture are mainly from this book.
- Free to read online at https://abseil.io/resources/swe-book

Implementation

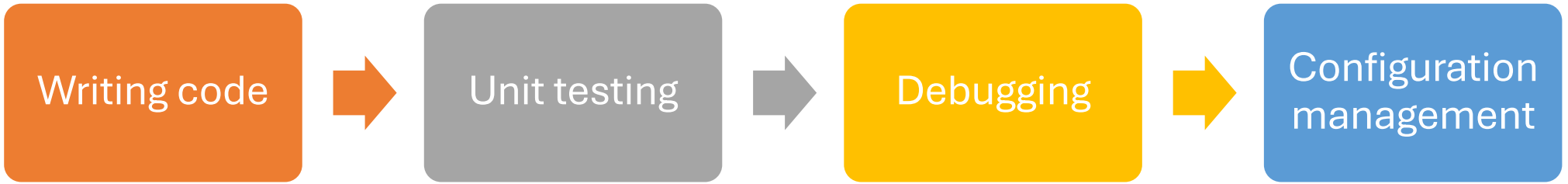
" Any fool can write code that a computer can understand. Good programmers write code that humans can understand."

M. Fowler (1999)



Implementation

- Transforming detailed designs into a valid program

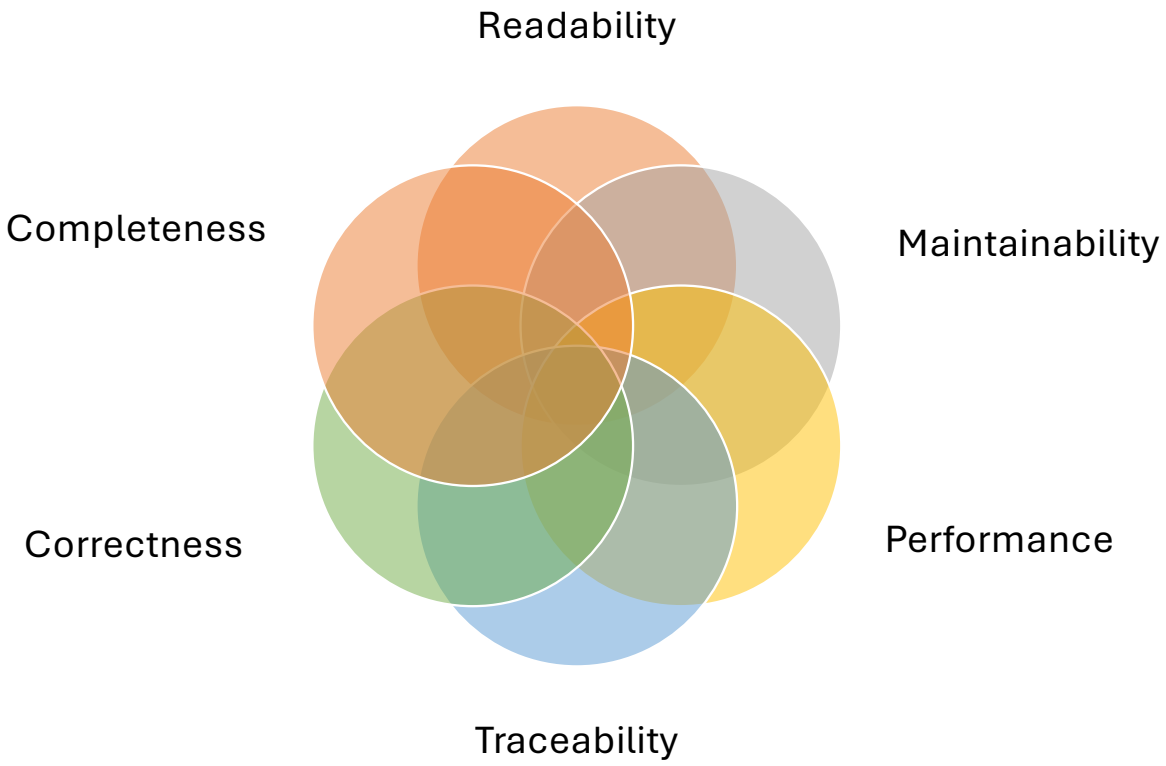




Mahidol University
Faculty of Information
and Communication Technology

Image generated by Bing

Good Implementations





Mahidol University
Faculty of Information
and Communication Technology



Style Guides and Rules

Software Engineering at Google
Chapter 8

Style Guides and Rules

- Most engineering organizations have rules governing their codebases
  - Rules about where source files are stored
  - Rules about the formatting of the code
  - Rules about naming and patterns and exceptions and threads.
- Most software engineers are working within the bounds of a set of policies that control how they operate.
- Organization should maintain a set of style guides that define their rules.

Why Have Rules?

- The goal of having rules in place is to encourage **"good" behavior** and discourage **"bad" behavior.**
- The interpretation of "good" and "bad" varies by organization, depending on what the organization cares about.
  - "good" might promote usage patterns that support a small memory footprint or prioritize potential runtime optimizations.
  - "good" might promote choices that exercise new language features.
  - Sometimes, an organization cares most deeply about consistency, so that anything inconsistent with existing patterns is "bad."



The Syle Guide

Rules to avoid dangers

Rules to enforce best practices

Rules to ensure consistency

Rules to avoid dangers

- How to use static members and variables
- Using lambda expressions
- Handling exceptions
- Building for threading, access control, and class inheritance.
- Standard vocabulary types that may be used and for what purposes.
- Hard-to-use and the hard-to-use-correctly



6.2 Caught exceptions: not ignored

Except as noted below, it is very rarely correct to do nothing in response to a caught exception. (Typical responses are to log it, or if it is considered "impossible", rethrow it as an `AssertionError`.)

When it truly is appropriate to take no action whatsoever in a catch block, the reason this is justified is explained in a comment.

```
try {
  int i = Integer.parseInt(response);
  return handleNumericResponse(i);
} catch (NumberFormatException ok) {
  // it's not numeric; that's fine, just continue
}
return handleTextResponse(response);
```

Exception: In tests, a caught exception may be ignored without comment *if* its name is or begins with `expected`. The following is a very common idiom for ensuring that the code under test *does* throw an exception of the expected type, so a comment is unnecessary here.

```
try {
  emptyStack.pop();
  fail();
} catch (NoSuchElementException expected) {
}
```

https://google.github.io/styleguide/javaguide.html#s6.2-caught-exceptions

Rules to enforce best practices

- These rules help keep the codebase healthy and maintainable.
  - For example, specify where and how code authors must include comments.

Comments

Comments are absolutely vital to keeping our code readable. The following rules describe what you should comment and where. But remember: while comments are very important, the best code is self-documenting. Giving sensible names to types and variables is much better than using obscure names that you must then explain through comments.

When writing your comments, write for your audience: the next contributor who will need to understand your code. Be generous — the next one may be you!

Comment Style

Use either the `//` or `/* */` syntax, as long as you are consistent.

You can use either the `//` or the `/* */` syntax; however, `//` is *much* more common. Be consistent with how you comment and what style you use where.

File Comments

Start each file with license boilerplate.

If a source file (such as a `.h` file) declares multiple user-facing abstractions (common functions, related classes, etc.), include a comment describing the collection of those abstractions. Include enough detail for future authors to know what does not fit there. However, the detailed documentation about individual abstractions belongs with those abstractions, not at the file level.

For instance, if you write a file comment for `frobber.h`, you do not need to include a file comment in `frobber.cc` or `frobber_test.cc`. On the other hand, if you write a collection of classes in `registered_objects.cc` that has no associated header file, you must include a file comment in `registered_objects.cc`.

https://google.github.io/styleguide/cppguide.html#Comments

Rules to ensure consistency

- Engineers writing code and others reading it can focus on what's getting done rather than how it is presented.
- **Benefits of code consistency**
  1. Consistency enables scaling – easier to build tools to help developers.
  2. Consistency helps for human scaling – better mobility across projects.
  3. Consistency ensures resilience to time.

4.1.3 Empty blocks: may be concise

An empty block or block-like construct may be in K & R style (as described in Section 4.1.2). Alternatively, it may be closed immediately after it is opened, with no characters or line break in between (`{}`), **unless** it is part of a *multi-block statement* (one that directly contains multiple blocks: `if/else` or `try/catch/finally`).

Examples:

```
// This is acceptable
void doNothing() {}

// This is equally acceptable
void doNothingElse() {
}
```

```
// This is not acceptable: No concise empty blocks in a multi-block statement
try {
  doSomething();
} catch (Exception e) {}
```

4.2 Block indentation: +2 spaces

Each time a new block or block-like construct is opened, the indent increases by two spaces. When the block ends, the indent returns to the previous indent level. The indent level applies to both code and comments throughout the block. (See the example in Section 4.1.2, Nonempty blocks: K & R Style.)

https://google.github.io/styleguide/javaguide.html

Good Implementation Practices

Keep functions small and focused

One function should do one thing. If you need to scroll to read it, it is doing too much. (See: Extract Method refactoring)

Do not repeat yourself (DRY)

Duplicated logic is a common source of bugs. If the same fix is needed in three places, someone will update two and miss the third.

Write code for the reader, not the compiler

The compiler does not care about names or structure. Your teammate fixing a bug at 2am does.

Handle errors explicitly

Do not silently ignore exceptions or return null without documenting it. Every error path is a decision that should be visible.

Commit small and often

Small commits make code review easier, bugs easier to trace, and reduce the risk of losing work.

Test as you write, not after

Writing tests after the fact often means testing what the code does rather than what it should do.



Mahidol University
Faculty of Information
and Communication Technology



Code Comments

Best Practices for Writing Code Comments

THE OVERFLOW

Essays, opinions, and advice on the act of computer programming from Stack Overflow.

Find something

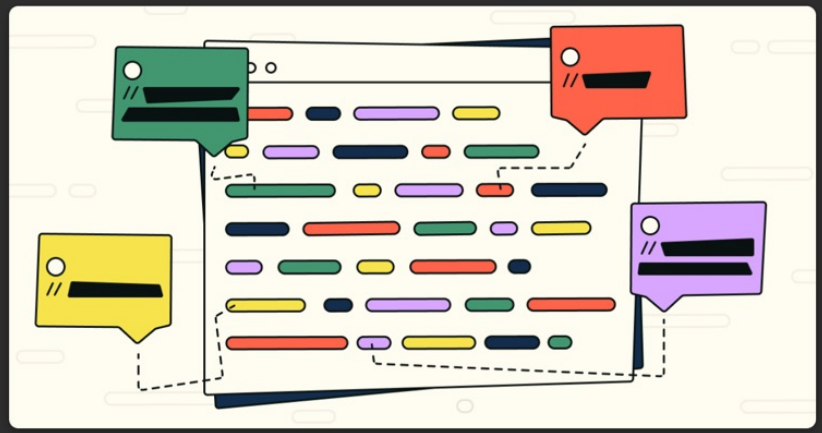
Latest Newsletter Podcast Company

code-for-a-living JULY 5, 2021

Best practices for writing code comments

While there are many resources to help programmers write better code—such as books and static analyzers—there are few for writing better comments. While it's easy to measure the quantity of comments in a program, it's hard to measure the quality, and the two are not necessarily correlated. A bad comment is worse than no comment at all. Here are some rules to help you achieve a happy medium.

Ellen Spertus



Ellen Spertus, https://stackoverflow.blog/2021/07/05/best-practices-for-writing-code-comments/

Best Practices for Writing Code Comments

Rule 1: Comments should not duplicate the code.

Rule 2: Good comments do not excuse unclear code.

Rule 3: If you can't write a clear comment, there may be a problem with the code.

Rule 4: Comments should dispel confusion, not cause it.

Rule 5: Explain unidiomatic code in comments.

Rule 6: Provide links to the original source of copied code.

Rule 7: Include links to external references where they will be most helpful.

Rule 8: Add comments when fixing bugs.

Rule 9: Use comments to mark incomplete implementations.

Ellen Spertus, https://stackoverflow.blog/2021/07/05/best-practices-for-writing-code-comments/

Best Practices for Writing Code Comments

Rule 1: Comments should not duplicate the code.

```
if (x > 3) {
    …
} // if
```

```
i = i + 1;          // Add one to i
```

```
// create a for loop // <-- comment
for // start for loop
(   // round bracket
    // newline
int // type for declaration
i    // name for declaration
=   // assignment operator for declaration
0   // start value for i
```

Ellen Spertus, https://stackoverflow.blog/2021/07/05/best-practices-for-writing-code-comments/

Best Practices for Writing Code Comments

Rule 2: Good comments do not excuse unclear code.

```
private static Node getBestChildNode(Node node) {
    Node n; // best child node candidate
    for (Node node: node.getChildren()) {
        // update n if the current state is better
        if (n == null || utility(node) > utility(n)) {
            n = node;
        }
    }
    return n;
}
```

```
private static Node getBestChildNode(Node node) {
    Node bestNode;
    for (Node currentNode: node.getChildren()) {
        if (bestNode == null || utility(currentNode) > utility(bestNode)) {
            bestNode = currentNode;
        }
    }
    return bestNode;
}
```

Ellen Spertus, https://stackoverflow.blog/2021/07/05/best-practices-for-writing-code-comments/

Best Practices for Writing Code Comments

Rule 3: If you can't write a clear comment, there may be a problem with the code.

This brings to mind Kernighan's Law:

> Debugging is twice as hard as writing the code in the first place. Therefore, if you write the code as cleverly as possible, you are, by definition, not smart enough to debug it.

Ellen Spertus, https://stackoverflow.blog/2021/07/05/best-practices-for-writing-code-comments/

Best Practices for Writing Code Comments

Rule 4: Comments should dispel confusion, not cause it.

No discussion of bad comments would be complete without this story from Steven Levy's *Hackers: Heroes of the Computer Revolution*:

> [Peter Samson] was particularly obscure in refusing to add comments to his source code explaining what he was doing at a given time. One well-distributed program Samson wrote went on for hundreds of assembly-language instructions, with only one comment beside an instruction that contained the number 1750. The comment was RIPJSB, and people racked their brains about its meaning until someone figured out that 1750 was the year Bach died, and that Samson had written an abbreviation for Rest In Peace Johann Sebastian Bach.

Ellen Spertus, https://stackoverflow.blog/2021/07/05/best-practices-for-writing-code-comments/

Best Practices for Writing Code Comments

Rule 5: Explain unidiomatic code in comments.

```
final Object value = (new JSONTokener(jsonString)).nextValue();
// Note that JSONTokener.nextValue() may return
// a value equals() to null.
if (value == null || value.equals(null)) {
    return null;
}
```

Ellen Spertus, https://stackoverflow.blog/2021/07/05/best-practices-for-writing-code-comments/

Best Practices for Writing Code Comments

```
/** Converts a Drawable to Bitmap. via https://stackoverflow.com/a/46018816/2219998. */
```

- what problem was being solved
- who provided the code
- why the solution is recommended
- what commenters thought of it
- whether it still works
- how it can be improved

Rule 6: Provide links to the original source of copied code.

```
// Many thanks to Chris Veness at http://www.movable-type.co.uk/scripts/latlong.html
// for a great reference and examples.
```

Ellen Spertus, https://stackoverflow.blog/2021/07/05/best-practices-for-writing-code-comments/

Best Practices for Writing Code Comments

```
// http://tools.ietf.org/html/rfc4180 suggests that CSV lines
// should be terminated by CRLF, hence the \r\n.
csvStringBuilder.append("\r\n");
```

Rule 7: Include links to external references where they will be most helpful.

Ellen Spertus, https://stackoverflow.blog/2021/07/05/best-practices-for-writing-code-comments/

Best Practices for Writing Code Comments

```
// NOTE: At least in Firefox 2, if the user drags outside of the browser window,
// mouse-move (and even mouse-down) events will not be received until
// the user drags back inside the window. A workaround for this issue
// exists in the implementation for onMouseLeave().
@Override
public void onMouseMove(Widget sender, int x, int y) { .. }
```

```
// Use the name as the title if the properties did not include one (issue #1425)
```

Rule 8: Add comments when fixing bugs.

Ellen Spertus, https://stackoverflow.blog/2021/07/05/best-practices-for-writing-code-comments/

Best Practices for Writing Code Comments

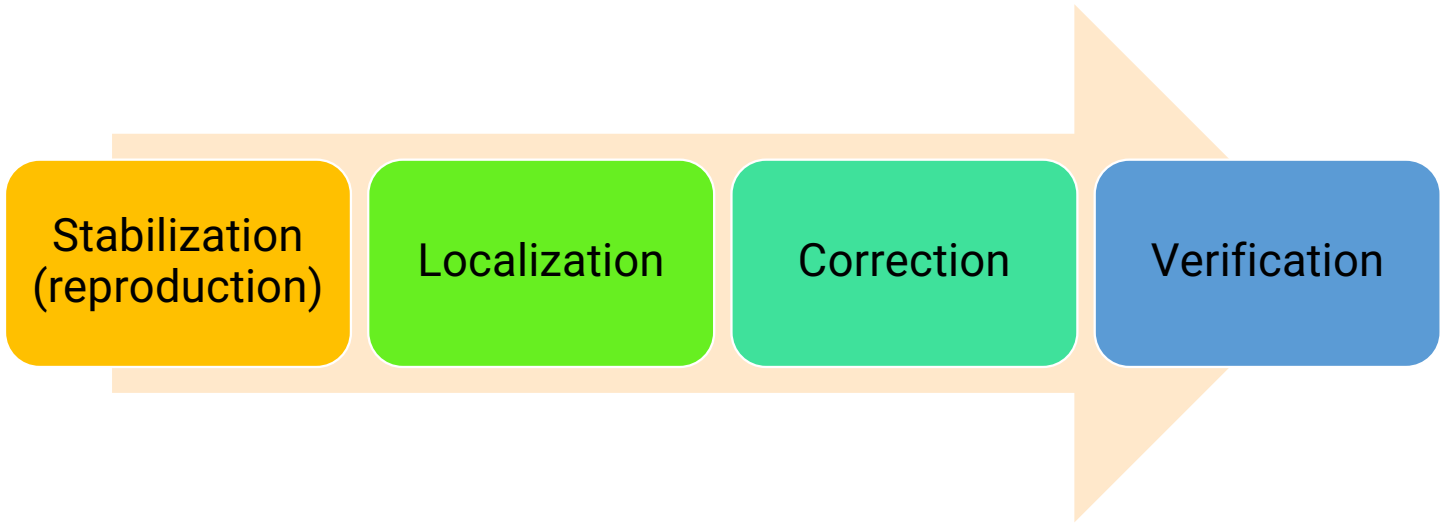
```
// TODO(hal): We are making the decimal separator be a period,
// regardless of the locale of the phone. We need to think about
// how to allow comma as decimal separator, which will require
// updating number parsing and other places that transform numbers
// to strings, such as FormatAsDecimal
```

Rule 9: Use comments to mark incomplete implementations.

Ellen Spertus, https://stackoverflow.blog/2021/07/05/best-practices-for-writing-code-comments/

Debugging

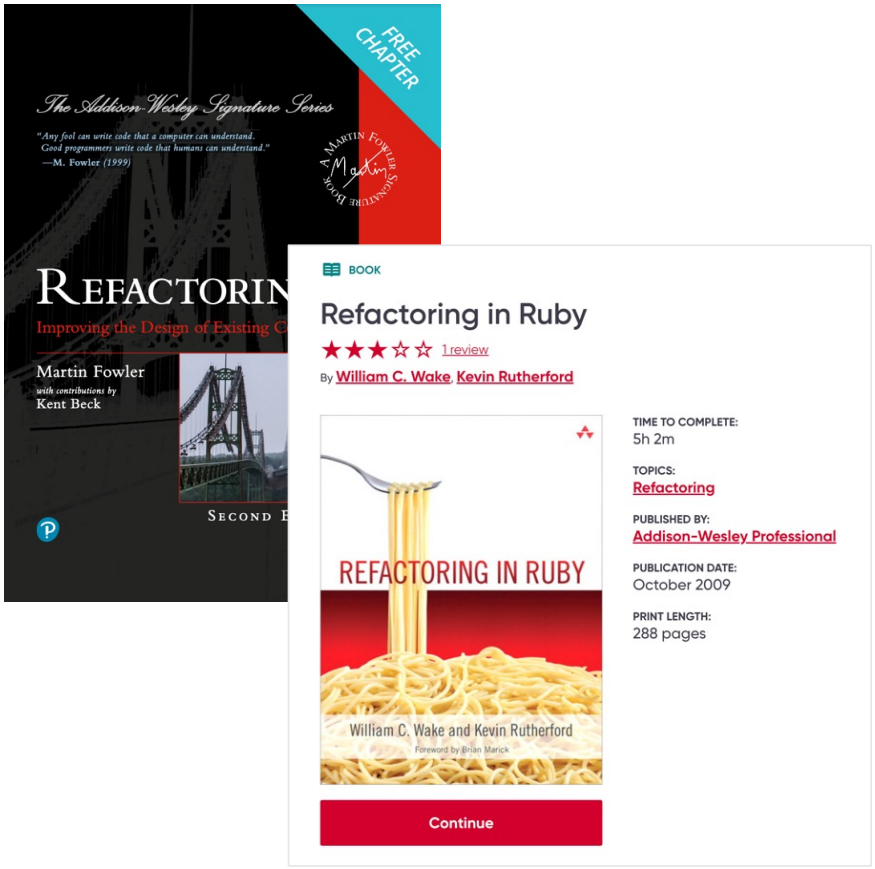
- Locating and fixing errors in code
- Errors noticed by testing, inspection, use
- Four phases



Debugging II

- Heuristics
  - Some routines will have many errors
  - Routines with an error tend to have more
  - New code tends to have more error
  - Particular ones: languages, parts, coders
- Tools
  - Code comparators
  - Extended checkers (lint)
  - Interactive debuggers
  - Special libraries
  - Others: profilers, pre/post conditions, test coverage

References for Today (2)



- Refactoring 2nd Edition by Martin Fowler (one chapter is free to read: https://martinfowler.com/books/refactoring.html) .
- Refactoring in Ruby by Wake and Rutherford. You can read this one from O'Reilly Online (https://www.li.mahidol.ac.th/e-books/#O)

What is Refactoring?

Refactoring is **the art of safely improving the design of existing code.**

"Refactoring is the process of changing a software system in such a way that it does not alter the external behavior of the code yet improves its internal structure." – Martin Folwer

A Few Implications

Refactoring does not include just any changes in a system.

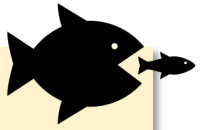
Refactoring is not rewriting from scratch.

Refactoring is not just any restructuring intended to improve code.

Refactoring supports emergent design.

Refactorings can be small or large.

Refactoring



Code Smells

- Duplicated code
- Long method
- Large class
- Switch statement
- Feature envy
- Intimacy

Refactoring Actions

- Extract method
- Substitute algorithm
- Move method
- Extract class
- … and many more

See more in the online catalog of refactoring actions: https://refactoring.com/catalog/

Refactoring

```
1  void renderBanner() {
2    if ((platform.toUpperCase().indexOf("MAC") > -1) &&
3         (browser.toUpperCase().indexOf("IE") > -1) &&
4          wasInitialized() && resize > 0 )
5    {
6      // do something
7    }
8  }
```

```
1  void renderBanner() {
2    final boolean isMacOs = platform.toUpperCase().indexOf("MAC") > -1;
3    final boolean isIE = browser.toUpperCase().indexOf("IE") > -1;
4    final boolean wasResized = resize > 0;
5
6    if (isMacOs && isIE && wasInitialized() && wasResized) {
7      // do something
```

Refactoring

```
1  int basePrice = quantity * itemPrice;
2  double seasonDiscount = this.getSeasonalDiscount();
3  double fees = this.getFees();
4  double finalPrice = discountedPrice(basePrice, seasonDiscount, fees);
```

```
1  int basePrice = quantity * itemPrice;
2  double finalPrice = discountedPrice(basePrice);
```

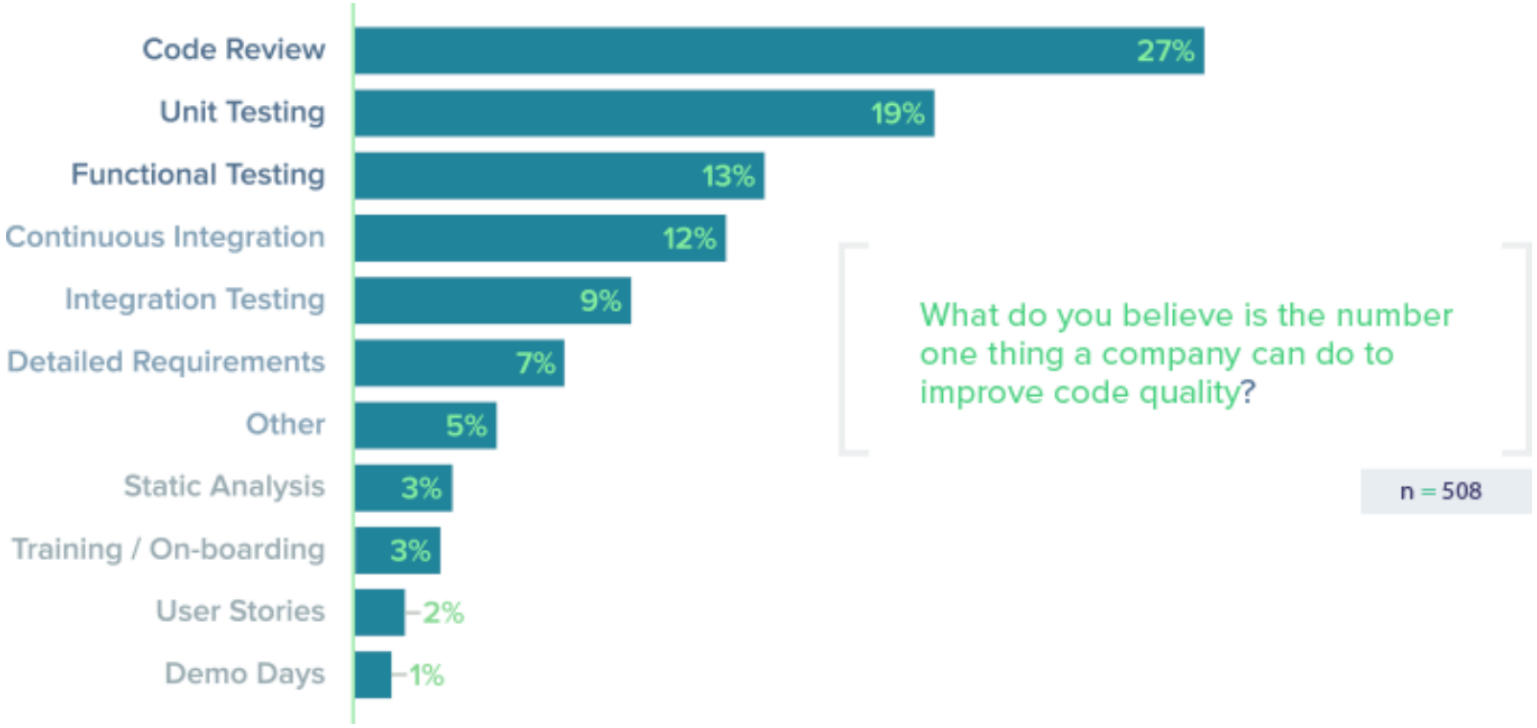


Mahidol University
Faculty of Information
and Communication Technology



Code Review

Common Practices for Writing Clean, Good Quality Code

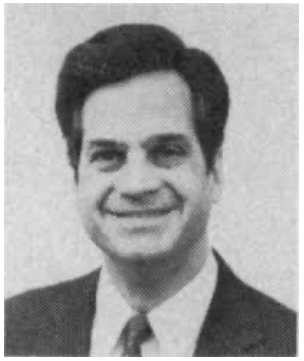


Widely used steps to improve code quality

https://medium.com/better-programming/good-code-vs-bad-code-35624b4e91bc

Code Inspection

- An idea of code inspection has been proposed by Fagan 1976
  - To improve software quality
  - To increase programmer productivity
- It is effective at finding defects and widely used at IBM
- It can be done on code, design artifacts, requirements, user information, and documentation, test plans and test cases.



Michael Fagan, IBM



Michael E. Fagan. Design and Code Inspections to Reduce Errors in Program Development. IBM Systems Journal, 1976.
Michale E. Fagan. A History of Software Inspection. sd&m Conference 2001, Software Pioneer, 2001 (http://www.mfagan.com/pdfs/software_pioneers.pdf)
Business vector created by freepik - www.freepik.com

designed by freepik.com

Code Inspection

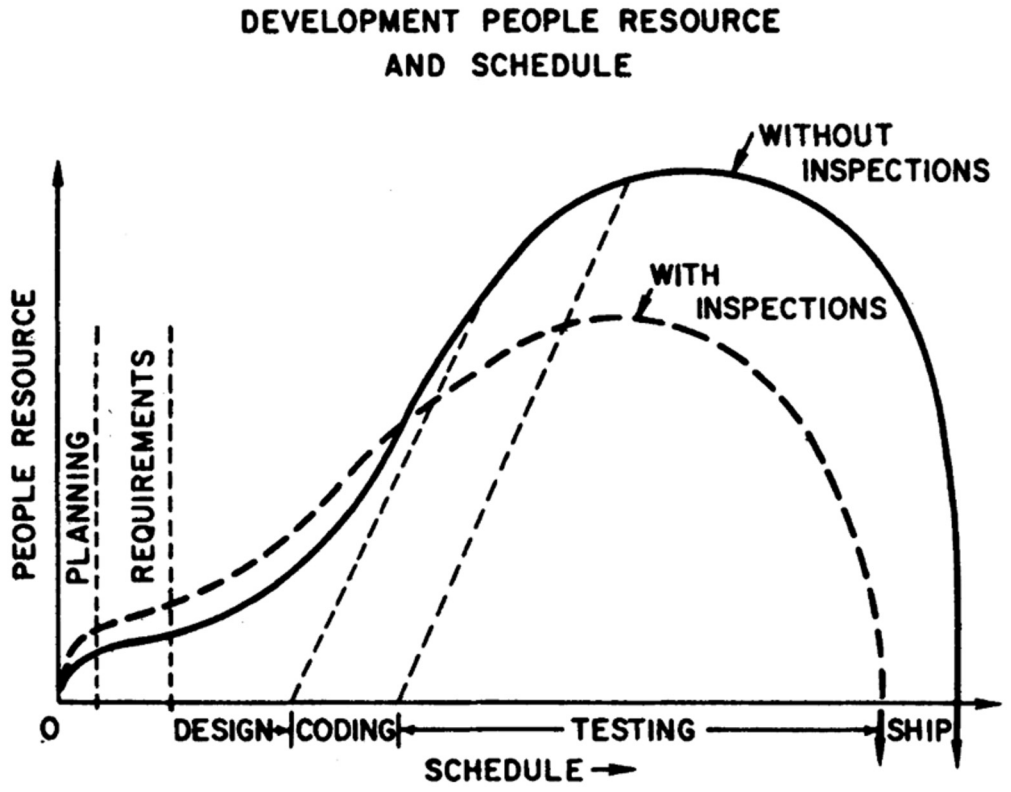


Fig. 1.

THE SOFTWARE QUALITY PROBLEM

Fagan, M. E. (1986). Advances in software inspections. IEEE Transactions on Software Engineering, SE-12(7), 744–751. https://doi.org/10.1109/TSE.1986.6312976

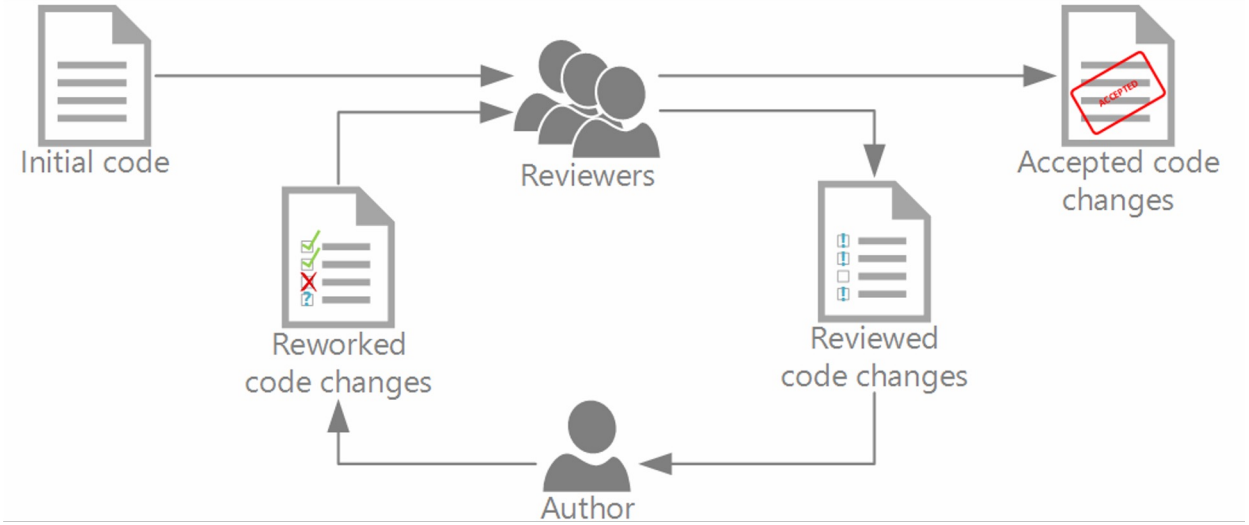
Inspection Process

Operation	Objective
PLANNING	• Materials to be inspected must meet inspection entry criteria. • Arrange the availability of the right participants. • Arrange suitable meeting place and time.
OVERVIEW	• Group education of participants in what is to be inspected. • Assign inspection roles to participants.
PREPARATION	• Participants learn the material and prepare to fulfill their assigned roles.
INSPECTION	• Find defects. (Solution hunting and discussion of design alternatives is discouraged.)
REWORK	• The author reworks all defects.
FOLLOW-UP	• Verification by the inspection moderator or the entire inspection team to assure that all fixes are effective and that no secondary defects have been introduced.

Fagan, M. E. (1986). Advances in software inspections. IEEE Transactions on Software Engineering, SE-12(7), 744–751. https://doi.org/10.1109/TSE.1986.6312976

Modern Code Review

- Unlike the code inspection technique, modern code reviews are conducted within modern code review tools.
- Various kinds of modern code review tools are available these days.



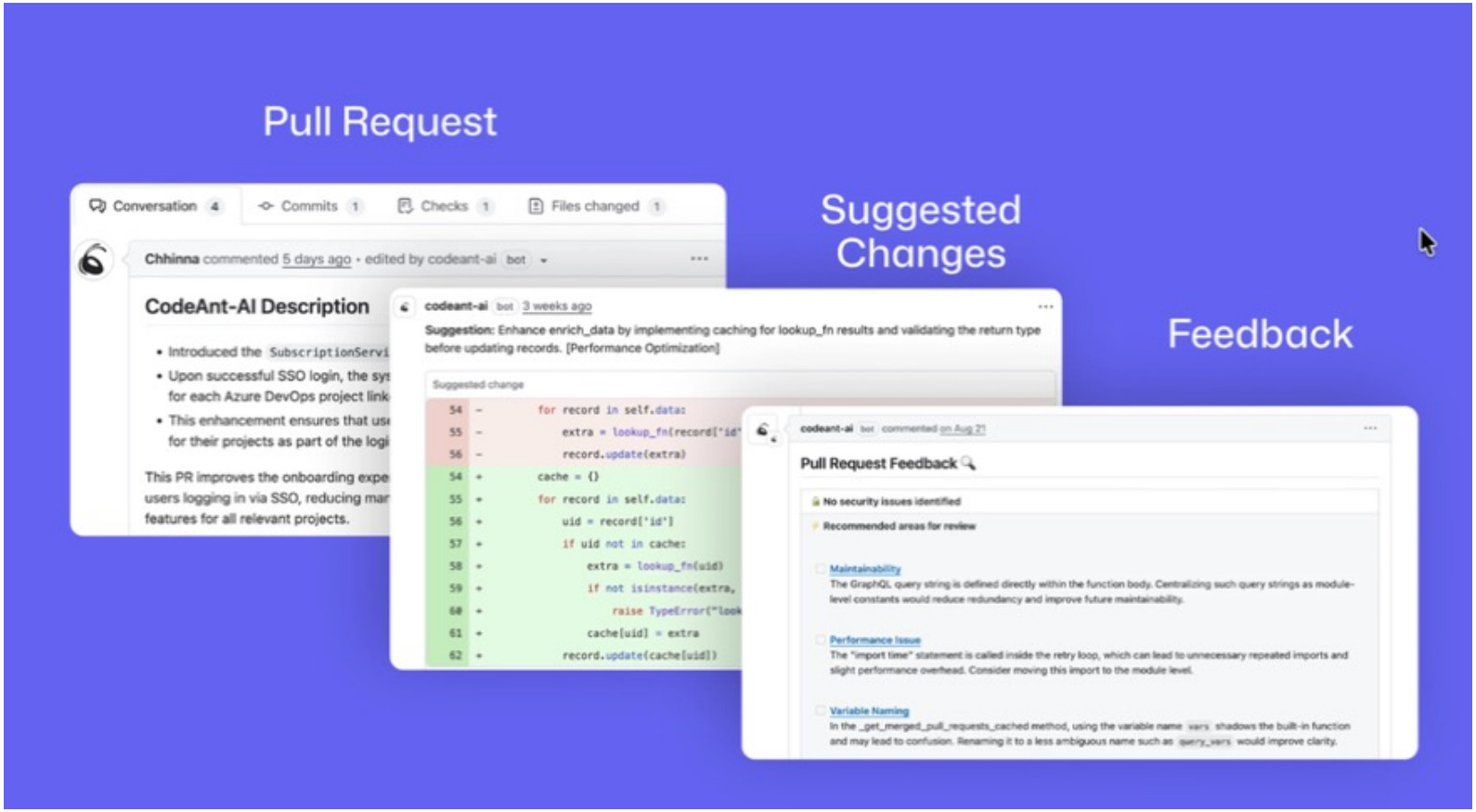
GitHub

Gerrit Code Review

PHABRICATOR

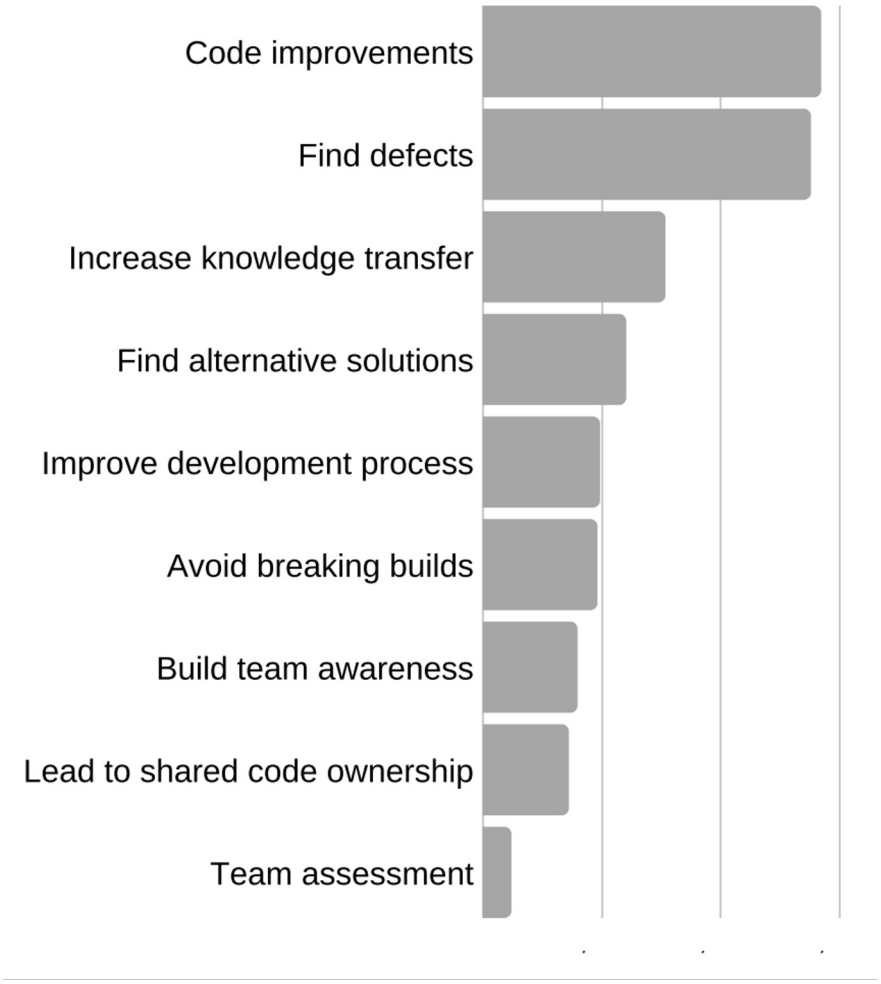
https://causecode.com/things-to-look-for-in-a-code-review/

Example



https://codeant.ai/blogs/good-code-review-practices-guide

Code Review Motivation



Michaela, G. Code Reviews (2019)

Benefits of Code Review (Google)

A well-designed code review process and a culture of taking code review seriously provides the following benefits:

1. Checks code correctness
2. Ensures the code change is comprehensible to other engineers
3. Enforces consistency across the codebase
4. Psychologically promotes team ownership
5. Enables knowledge sharing
6. Provides a historical record of the code review itself

Code Review Checklist

Implementation

- Does this code change do what it is supposed to do?
- Can this solution be simplified?
- Does this change add unwanted compile-time or run-time dependencies?

Dependencies

- If this change requires updates outside of the code, like updating the documentation, configuration, readme files, was this done?
- Might this change have any ramifications for other parts of the system, backward compatibility?

Code Review Checklist (cont.)

Security and Data Privacy

- Does this code open the software for security vulnerabilities?
- Are authorization and authentication handled in the right way?
- Is sensitive data like user data, credit card information securely handled and stored?

Logic Errors and Bugs

- Can you think of any use case in which the code does not behave as intended?
- Can you think of any inputs or external events that could break the code?

Code Review Checklist (cont.)

Error Handling and Logging

- Is error handling done the correct way?
- Should any logging or debugging information be added or removed?
- Are error messages user-friendly?

Testing and Testability

- Is the code testable?
- Does it have enough automated tests (unit/integration/system tests)?

Readability

- Was the code easy to understand?
- Which parts were confusing to you and why?
- Can the readability of the code be improved by smaller methods?

Code Review Checklist (cont.)

Usability and Accessibility

- Is the proposed solution well designed from a usability perspective? Is the API well documented?

Performance

- Do you think this code change will impact system performance in a negative way?
- Do you see any potential to improve the performance of the code?

Experts Opinion

- Do you think a specific expert, like a security expert or a usability expert, should look over the code before it can be committed?
- Will this code change impact different teams?
- Should they have a say on the change as well?

Discussion: Code Review in Practice

Scenario

A junior developer submits a pull request to fix a bug. The code works correctly. However, the reviewer notices:

- A variable named x stores the user's account balance
- No comments explain why the fix works
- The same logic appears in two other methods
- No unit test was added for the bug fix

Answer these questions

1. Should the reviewer approve this pull request? Why or why not?
2. Which items in the code review checklist apply here?
3. How would you write the review comment to the developer? What tone would you use?
4. What is the minimum the developer must fix before the PR can be approved?

Conclusion

Good implementation is more than writing code that runs. It is about writing code that others can read, maintain, and trust.

- Good implementations balance readability, correctness, maintainability, performance, traceability, and completeness
- Style guides and coding rules exist to keep a codebase consistent and manageable across teams
- Comments should explain intent, not just repeat the code
- Refactoring improves the internal structure of code without changing its behavior, keeping code clean as it evolves
- Debugging is a systematic process: stabilize, localize, correct, and verify
- Code review is a shared responsibility, it catches bugs, spreads knowledge, and builds team ownership of the codebase

- Two questions every developer should ask before committing code:
  - Would I be comfortable if my team lead read this right now?
  - Could I understand this code six months from now with no context?